Future Software Engineers - FSE K02

Độ phức tạp tính toán và Array

Pham Ngoc Lam, Feb 2022.

Contents

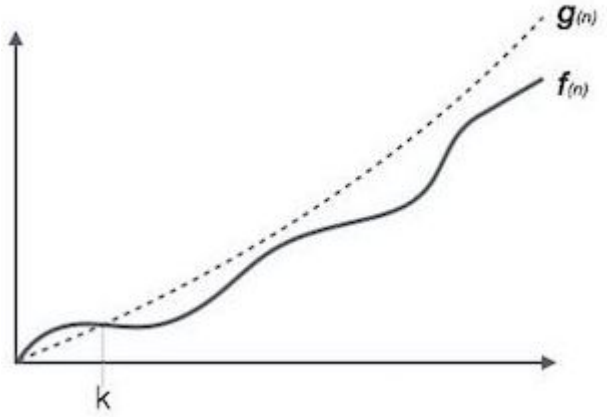
1. Độ phức tạp tính toán
2. Array là gì?
3. Livecoding: Examples
4. Homework exercises

1. Độ phức tạp tính toán

1.1. Độ phức tạp tính toán là gì?

- Dùng để đánh giá độ tốt của một thuật toán
- Thông thường gồm 2 loại chính

1. Time complexity (thời gian chạy)
2. Space complexity (độ phức tạp không gian)



1.2. Các khái niệm toán học cơ bản

- Big O(O)
  - Mô tả hàm bằng ký hiệu O lớn thường chỉ cung cấp một chặn trên cho tốc độ tăng của hàm.
  - Nếu f(x) = O(g(x)) khi x -> a thì:

$$\limsup_{x \to a} \left| \frac{f(x)}{g(x)} \right| < \infty.$$

  - Ví dụ: Xét x ra +oo có :
    - Nếu f(x) = 10, g(x) = 1 thì f(x) = O(g(x))
    - Nếu f(x) = 10 * x ^ 3, g(x) = x ^ 4 thì f(x) = O(g(x)) nhưng g(x) != O(f(x))

1.3. Time Complexity và Space complexity

- Được đại diện bởi 1 hàm số (1 hoặc nhiều biến) để ước lượng thời gian chạy và không gian nhớ của thuật toán.

```
def get_sum():
    #Time Complexity: O(10) - thuật toán sử dụng khoảng 10 phép tính
    #Space Complexity: O(2) - thuật toán phải allocate bộ nhớ cho 2 biến
    total_sum = 0
    for i in range(10):
        total_sum += i
    return total_sum

def get_summ_ver_2(n):
    #Time Complexity: O(n) - thuật toán sử dụng khoảng n phép tính
    #Space Complexity: O(2) - thuật toán phải allocate bộ nhớ cho 2 biến
    total_sum = 0
    for i in range(n):
        total_sum += i
    return total_sum
```

1.3. Time Complexity và Space complexity

QUIZ: Time and Space Complexity of the code snippet below

```python
def get_sum(n):
    total = 0
    for i in range(1, n):
        for j in range(i, n):
            total += i * j
    return total


def get_sum_v2(n):
    total_list = []
    for i in range(1, n):
        for j in range(1, n):
            total_list.append(i * j)
    ans = 0
    for value in total_list:
        ans += value
    return ans
```

2. Array

2.1. Array?

1. Gồm có 2 loại
   - Static Array
   - Dynamic Array

2.2. Static Array

- Static arrays are allocated memory at compile time for popular PLs (C++, Java…).
- Size is fixed.
- Located in stack memory space normally.
- Eg. : int array[10]; //array of size 10 in C++

2.2. Dynamic Array

- Memory is allocated at run time for popular PLs (Python, Java …).
- Size is not fixed.
- Located in heap memory space normally.
- Eg. : dynamic_arr = [10] * 100 (python)
- Các thao tác thông thường :
  1. Pop (pop 1 phần tử ở index bất kì)
  2. Append (thêm phần tử vào bên phải)

2.3. Time complexity (Tham khảo)

Parameter	Linked list	Array	Dynamic array
Indexing	O(n)	O(1)	O(1)
Insertion/deletion at beginning	O(1)	O(n), if array is not full (for shifting the elements)	O(n)
Insertion at ending	O(n)	O(1), if array is not full	O(1), if array is not full O(n), if array is full
Deletion at ending	O(n)	O(1)	O(n)
Insertion in middle	O(n)	O(n), if array is not full (for shifting the elements)	O(n)
Deletion in middle	O(n)	O(n), if array is not full (for shifting the elements)	O(n)
Wasted space	O(n) (for pointers)	0	O(n)

Note: Deletion at ending O(1) for dynamic array normally.

3. Live Coding

3.1. Example

- https://leetcode.com/problems/merge-sorted-array/
- https://leetcode.com/problems/range-sum-query-immutable/
- https://leetcode.com/problems/sum-of-unique-elements/

Code mẫu: https://pastebin.pl/view/0f9296e7

4. Homework

4.1. Homework

- https://leetcode.com/problems/pascals-triangle/
- https://leetcode.com/problems/plus-one/
- https://leetcode.com/problems/remove-element/

Gitclass: (submit code)

Code mẫu: https://pastebin.pl/view/20398ee6

References

https://en.wikipedia.org/wiki/Big_O_notation